Manual Maestro: Edición de Video Programática y Agéntica con Claude Code

Nota de exportación a Word (.docx): Este documento está estructurado con jerarquía de encabezados, tablas y bloques técnicos compatibles con Microsoft Word y Google Docs. Puedes exportarlo o copiar su contenido directamente desde el panel del editor para compartirlo como manual corporativo o guía de estudio.

1. Introducción: El Paradigma del "Video como Código"

La edición de video con **Claude Code** sustituye el trabajo manual de arrastrar y cortar clips con el ratón por un flujo de trabajo **programático y agéntico**. En este entorno, Claude Code actúa como un **Director Técnico y Editor Senior** que opera desde la terminal:

1. **Escucha y lee el material bruto** mediante transcripciones con marcas de tiempo a nivel de palabra (*word-level timestamps*) y mapas matemáticos de ondas de audio.
2. **Toma decisiones editoriales** eliminando silencios, muletillas y tomas falsas (frases repetidas) con precisión de milisegundos.
3. **Diseña gráficos animados (Motion Graphics)** programando componentes en React y TypeScript a $60fps$ mediante **Remotion**.
4. **Controla programas profesionales de edición (NLEs)** como **DaVinci Resolve** y **Adobe Premiere Pro** a través de servidores **MCP (Model Context Protocol)**, o renderiza directamente mediante **FFmpeg**.
5. **Se autoverifica** re-transcribiendo el video exportado e inspeccionando hojas de contacto visuales (*Contact Sheets*) para asegurar calidad de estudio.

2. Qué Descargar e Instalar (El Stack Base)

Para que Claude Code pueda analizar, cortar, diseñar y renderizar video en tu computadora local, debes instalar las siguientes herramientas base:

2.1 Herramientas de Sistema y Runtimes

Herramienta	Comando de Instalación Rápida	Función en el Flujo de Edición
Claude Code CLI	npm install -g @anthropic-ai/claude-code	El agente central que orquesta scripts, plugins, subagentes y servidores MCP.

Node.js (v20 / v22 LTS)	winget install OpenJS.NodeJS.LTS (Win) / brew install node (Mac)	Ejecuta el motor visual **Remotion** y los servidores MCP basados en TypeScript.
Python (3.10 +) y uv	curl -LsSf https://astral.sh/uv/install.sh | sh (Mac/Linux) o powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex" (Win)	Gestiona entornos virtuales ultrarrápidos para análisis de audio, IA local y detección de escenas.
FFmpeg y FFprobe	winget install Gyan.FFmpeg (Win) / brew install ffmpeg (Mac)	El motor de procesamiento para cortar clips sin recodificar, normalizar audio a $-14LUFS$ y extraer fotogramas.
yt-dlp	uv tool install yt-dlp o brew install yt-dlp	Descarga videos de referencia, extrae B-roll y permite analizar matemáticamente el estilo de otros editores.

2.2 Motores Locales de IA y Video (Instalados por Proyecto)

- **Faster-Whisper (Python):** Modelo local gratuito de reconocimiento de voz que transcribe el audio generando archivos JSON con la marca de tiempo exacta (en milisegundos) de cada palabra pronunciada.
- **PySceneDetect (Python):** Detecta cambios de plano y cortes visuales en videos existentes para extraer métricas de ritmo (*Taste Profiles*).
- **Remotion CLI:** Motor de video basado en React (npx create-video@latest) que permite previsualizar animaciones, subtítulos dinámicos y composiciones multicapa en el navegador en tiempo real.

3. Catálogo Completo de Complementos (Plugins, Skills y MCP Servers)

El ecosistema de edición en Claude Code se compone de tres capas: **Plugins y Skills** (conocimiento técnico e instrucciones de código), **Servidores MCP para Editores Tradicionales** y **Servidores MCP de IA Generativa**.

3.1 Plugins Oficiales y Skills de Código Abierto

Complemento	Comando de Instalación / Origen	Para qué sirve exactamente
Remotion Official Plugin	claude plugin marketplace add remotion-dev/claude-code-plugin claude plugin install remotion@remotion	Permite a Claude diseñar *lower-thirds*, pantallas divididas, zooms dinámicos y subtítulos estilo TikTok/Reels en React. Añade el comando /remotion:remotion-create.
Remotion Agent Skills	npx remotion skills add (ejecutar dentro de la carpeta Remotion)	Inyecta en el proyecto las reglas exactas de la API de Remotion (useCurrentFrame(), interpolate(), spring()) para evitar que Claude utilice animaciones CSS incompatibles con el renderizado por fotogramas.
Claude Code Video Toolkit	git clone https://github.com/digitalsamba/claude-code-video-toolkit	Entorno de producción integral con comandos /setup, /video, /design y /brand. Automatiza desde el guion, locución TTS, captura de demos web con Playwright y diseño visual por actos.

Proofcut (Plugin + MCP)	Repositorio Open Source proofcut	Especializado en edición automática de *talking-head* (persona hablando a cámara). Corta tomas falsas basándose en el texto, inserta B-roll bajo líneas de diálogo específicas y se autocorrige re-transcribiendo el render final.
Professional Video Editing Skill	npx skillfish add edward-h26/onepromptclaudecode video-editing	Genera Listas de Decisiones de Edición (EDL), automatiza filtros complejos de FFmpeg (filter_complex) y aplica reencuadre automático ($9{:}16$, $1{:}1$, $16{:}9$).

3.2 Servidores MCP para Editores Tradicionales (NLEs)

Si prefieres que Claude Code haga todo el trabajo pesado de limpieza, ordenamiento y primer corte, pero te entregue una **línea de tiempo editable** para dar el toque artístico final:

1. **DaVinci Resolve MCP / Scripting Bridge:**
   - **Qué hace:** Conecta Claude Code con la API local de DaVinci Resolve Studio.
   - **Utilidad:** Crea el proyecto, importa el material bruto al *Media Pool*, ejecuta el *rough cut* (elimina silencios y tomas repetidas), aplica un LUT de color (.cube) en el árbol de nodos y deja la línea de tiempo abierta y lista para revisión humana.
2. **Adobe Premiere Pro MCP (leancoderkavy/premiere-pro-mcp):**
   - **Qué hace:** Conecta Claude Code con Premiere Pro mediante un puente de extensión CEP/UXP.
   - **Utilidad:** Permite ordenar por consola: *"Coloca todos los clips limpios de A-Roll en la pista V1 según el guion, aplica cortes en J de* $150ms$*, coloca la música en A2 con auto-ducking y añade marcadores verdes donde deba ir un gráfico"*.
3. **ChatCut MCP:**
   - **Qué hace:** Proporciona una interfaz de edición multipista basada en transcripción operada por Claude sin requerir licencias de software propietario.

3.3 Servidores MCP de IA Generativa y Clipping Viral

Servidor MCP	Configuración en Claude Code	Utilidad Práctica
OpusClip MCP	claude mcp add --transport http opusclip https://mcp.opus.pro/mcp	Analiza videos largos (podcasts, conferencias, streams), identifica los fragmentos de mayor retención, recorta en vertical $9:16$ con seguimiento facial (*Active Speaker Detection*) e incrusta subtítulos dinámicos.
OpenClip MCP	claude mcp add --transport http openclip https://openclip.app/mcp	Ofrece 32 herramientas para recorte inteligente, eliminación de fondos en video y generación de B-roll contextual.
ElevenLabs MCP	npx -y @elevenlabs/mcp-server (requiere ELEVENLABS_API_KEY)	Genera efectos de sonido a medida (*whooshes*, *risers*, impactos cinemáticos) para transiciones, locuciones o corrección de palabras mal pronunciadas mediante clonación de voz.
Shotstack / Hyperframes MCP	API Cloud JSON / MCP	Renderizado en la nube a gran escala y creación de animaciones de interfaz (UI motion graphics) para tutoriales de software.

4. Estructura Profesional del Proyecto

El error más frecuente al editar video con IA es colocar archivos .mp4 pesados sin orden y pedirle a Claude que los procese a ciegas. Una estructura profesional **separa los archivos multimedia pesados de los metadatos ligeros en JSON**, permitiendo a Claude leer horas de video en pocos kilobytes de texto.

None

```
mi-estudio-video/
├── CLAUDE.md                     # El "Cerebro": Reglas de
estilo, marca, audio y flujo de trabajo
├── project.json                  # Estado del proyecto: escenas,
duración objetivo, checklist de fases
├── pyproject.toml                # Dependencias de Python
gestionadas con uv (faster-whisper, scenedetect)
├── .claude/
│   ├── settings.json             # Permisos preaprobados para
ejecutar ffmpeg, python y npm sin interrupciones
│   ├── agents/
│   │   ├── cutter-agent.md       # Subagente especialista en
corte híbrido (transcripción + onda de audio)
│   │   ├── motion-designer.md    # Subagente especialista en
animaciones React/Remotion a 60fps
│   │   └── qa-inspector.md       # Subagente de control de
calidad post-render
│   └── commands/
│       ├── rough-cut.md          # Comando (/rough-cut) para
limpiar silencios y tomas falsas
│       ├── verify-render.md      # Comando (/verify-render) para
auditoría visual y de transcripción
│       └── clone-style.md        # Comando (/clone-style) para
extraer el perfil de ritmo de un video de referencia
├── brand/
│   ├── brand.json                # Colores HEX, tipografías,
curvas Bezier y reglas de subtítulos
│   └── assets/                   # Logos en SVG, efectos de
sonido (.wav), LUTs de color (.cube)
├── media/
│   ├── 01_raw_aroll/             # Tomas originales hablando a
cámara (solo lectura, nunca se sobrescriben)
│   ├── 02_broll/                 # Clips de apoyo, grabaciones
de pantalla, recursos visuales
│   ├── 03_audio/                 # Música de fondo, pistas de
voz aisladas, SFX generados
│   └── 04_proxies/               # Copias ligeras en 480p/720p
generadas para previsualización rápida
```

```
├── data/                          # LA CAPA DE DATOS QUE CLAUDE
CODE LEE Y EDITA
|   ├── transcripts/               # Archivos JSON de
Faster-Whisper con timestamps por palabra
|   ├── waveforms/                 # Mapa de silencios y picos de
volumen en milisegundos
|   ├── style_profiles/            # Métricas extraídas de
creadores de referencia (WPM, ASL)
|   └── edl.json                   # Edit Decision List: mapa
exacto de cortes, zooms, B-roll y J-Cuts
├── remotion/                      # Motor visual en React +
TypeScript
|   ├── package.json
|   └── src/
|       ├── Root.tsx
|       ├── MainComposition.tsx    # Ensamblaje maestro
sincronizado con data/edl.json
|       └── components/            # DynamicCaptions.tsx,
LowerThird.tsx, SmoothZoom.tsx, BrollOverlay.tsx
├── scripts/                       # Herramientas locales
invocadas por Claude Code
|   ├── analyze_audio_hybrid.py    # Cruza Faster-Whisper + FFmpeg
silencedetect + detector de tomas falsas
|   ├── build_rough_cut.py         # Ejecuta el corte sin pérdidas
o genera XML para DaVinci/Premiere
|   ├── verify_render.py           # Re-transcribe el render y
genera hoja de contactos (.jpg)
|   └── extract_style.py           # Analiza videos de referencia
con PySceneDetect y Whisper
└── renders/
    ├── drafts/                    # Borradores rápidos + hojas de
contacto visuales para QA
    └── final/                     # Exportación maestra en 4K /
1080p (H.264 / ProRes)
```

5. Las 5 Técnicas Avanzadas para Resultados de Estudio

5.1 Corte Híbrido: Transcripción + Onda de Audio (*Waveform*)

Cortar un video guiándose únicamente por los tiempos que arroja un modelo de transcripción produce cortes robóticos y mutila el final de las palabras.

- **Cómo funciona:** El script analyze_audio_hybrid.py extrae primero las marcas de tiempo palabra por palabra con faster-whisper y simultáneamente ejecuta el filtro silencedetect=noise=-35dB:d=0.25 de FFmpeg.
- **Regla de Oro:** Claude nunca corta exactamente en el milisegundo donde termina la palabra según el texto; busca el valle de silencio real en la onda de audio e inyecta un **acolchado acústico (*padding*)** de $80ms$ antes del inicio de cada frase y $120ms$ después del final de la última palabra, aplicando además un micro-fundido de audio (afade) de $15ms$ para evitar chasquidos (*clicks*).

5.2 Detección Inteligente de Tomas Falsas (*Retake Elimination*)

Cuando grabas hablando a cámara y te equivocas, sueles repetir la frase desde el principio (por ejemplo: *"Hoy vamos a ver cómo... perdón... Hoy vamos a descubrir cómo editar con IA"*).

- **Cómo funciona:** El subagente cutter-agent analiza semánticamente los segmentos del JSON de transcripción. Cuando detecta dos o más oraciones consecutivas con un inicio léxico similar o separadas por palabras de reinicio (*"perdón"*, *"otra vez"*, *"tres, dos, uno"*), descarta automáticamente los intentos fallidos y conserva únicamente la última toma completa y fluida.

5.3 Bucle de Autoverificación Post-Render (Método *Proofcut* + Visión Multimodal)

Claude Code no reproduce video en vivo, pero puede auditar su propio trabajo combinando texto e imagen estática:

1. **Auditoría Acústica/Textual:** Al exportar un borrador en renders/drafts/draft_v1.mp4, Claude ejecuta nuevamente faster-whisper sobre el archivo exportado y compara la nueva transcripción contra el guion limpio esperado. Si detecta una frase repetida o una palabra cortada, ajusta los milisegundos en data/edl.json y vuelve a exportar.
2. **Auditoría Visual (Contact Sheet):** El script verify_render.py utiliza FFmpeg para extraer 1 fotograma por cada cambio de escena o cada 3 segundos y los ensambla en una cuadrícula de alta resolución (contact_sheet.jpg) con el código de tiempo impreso en cada miniatura. Claude inspecciona esa imagen con su capacidad de visión para comprobar que ningún subtítulo tape el rostro del presentador y que los textos respeten los márgenes de seguridad.

5.4 Cortes en J y en L (*J-Cuts & L-Cuts*) Programáticos

En el cine y la edición profesional, el audio y el video rara vez cortan en el mismo fotograma exacto:

- **J-Cut:** El audio de la siguiente escena comienza a escucharse entre $150ms$ y $250ms$ **antes** de que entre la nueva imagen.
- **L-Cut:** La voz de la escena actual continúa escuchándose sobre el inicio del clip de B-roll siguiente.

- **Implementación:** En data/edl.json, cada segmento define tiempos independientes para video_in/video_out y audio_in/audio_out, que Remotion o el generador XML de DaVinci/Premiere respetan automáticamente.

5.5 Clonación Matemática de Estilo (*Taste Profiles*)

Para replicar el dinamismo de tu creador de contenido favorito sin adivinar:

1. Descargas 3 videos de referencia con yt-dlp.
2. Ejecutas /clone-style, que mide con PySceneDetect y faster-whisper:
    - **ASL (*Average Shot Length*):** Duración media entre cambios visuales (por ejemplo, $4.2s$).
    - **WPM (*Words Per Minute*):** Velocidad de habla efectiva y duración media de las pausas ($0.18s$).
    - **Densidad de B-Roll:** Porcentaje del video cubierto por recursos visuales frente a cámara principal.
3. Estas métricas se guardan en data/style_profiles/referencia.json y se aplican como restricciones matemáticas al generar tu edl.json.

6. Flujo de Trabajo Paso a Paso (Manual de Operación Diaria)

1. **Inicialización del Entorno (Una sola vez por proyecto):** Coloca el archivo CLAUDE.md en la raíz de tu carpeta vacía, abre la terminal en esa ubicación, inicia claude y pídele:
   "Lee el archivo CLAUDE.md y ejecuta la Sección 0 de Bootstrap para construir toda la estructura del proyecto, instalar los plugins, configurar Python con uv e inicializar Remotion."
2. **Ingesta de Material:** Copia tus grabaciones originales de cámara en media/01_raw_aroll/ y tus clips de apoyo en media/02_broll/.
3. **Primer Corte Automático (Rough Cut Híbrido):** Ejecuta en Claude Code el comando:

```
None

/rough-cut media/01_raw_aroll/mi_toma.mp4
```

4. Claude analizará la voz y la onda de audio, eliminará silencios y tomas falsas, generará el data/edl.json y creará un borrador limpio o un XML listo para abrir en DaVinci Resolve / Premiere Pro.
5. **Diseño de Motion Graphics y Subtítulos:** Pídele a Claude Code:
   "Usando el subagente motion-designer y nuestro brand/brand.json, añade subtítulos dinámicos palabra por palabra en Remotion, inserta un lower-third en el segundo 00:08 y aplica zooms suaves del 100% al 112% en las frases marcadas como énfasis en edl.json."
6. **Control de Calidad y Render Maestro:** Ejecuta en Claude Code:

None

```
/verify-render renders/drafts/draft_v1.mp4
```

8.

9. Una vez que Claude verifique la hoja de contactos visual y la transcripción sin errores, ordénale exportar el master final en renders/final/ con audio normalizado a $-14 LUFS$.